

Exercise 1: Control Structures

Scenario 1: Apply 1% Discount to Loan Interest Rates:

```
DECLARE
BEGIN
    FOR c IN (SELECT CustomerID, Age FROM Customers) LOOP
        IF c.Age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE CustomerID = c.CustomerID;
            DBMS_OUTPUT.PUT_LINE('1% interest discount given to Customer ' || c.CustomerID);
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

Output:

1% interest discount given to Customer 101

1% interest discount given to Customer 104

Scenario 2: Set VIP Status

```
DECLARE
BEGIN
    FOR c IN (SELECT CustomerID, Balance FROM Customers) LOOP
        IF c.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP = 'TRUE'
            WHERE CustomerID = c.CustomerID;
            DBMS_OUTPUT.PUT_LINE('Customer ' || c.CustomerID || ' has been marked as VIP.');
```

/

Output:

Customer 102 has been marked as VIP.

Customer 105 has been marked as VIP.

Scenario 3: Loan Due Reminder

DECLARE

BEGIN

FOR I IN (

SELECT LoanID, CustomerID, DueDate

FROM Loans

WHERE DueDate >= SYSDATE

AND DueDate <= SYSDATE + 30

) LOOP

DBMS_OUTPUT.PUT_LINE(

'Reminder: Customer ' || I.CustomerID ||

', your Loan ID ' || I.LoanID ||

' is due on ' || TO_CHAR(I.DueDate, 'DD-MON-YYYY')

);

END LOOP;

END;

/

Output:

Reminder: Customer 101, your Loan ID 2001 is due on 20-JUN-2026

Reminder: Customer 105, your Loan ID 2005 is due on 28-JUN-2026

Exercise 3: Stored Procedures

Scenario 1: Process Monthly Interest

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (0.01 * Balance)
    WHERE AccountType = 'SAVINGS';
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Monthly interest credited to savings accounts.');
```

END;

/

EXECUTE: EXEC ProcessMonthlyInterest;

Output:

Monthly interest credited to savings accounts.

Scenario 2: Update Employee Bonus

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus
( p_dept VARCHAR2, p_bonus NUMBER)
AS
BEGIN
    UPDATE Employees
    SET Salary = Salary * (1 + p_bonus/100)
    WHERE Department = p_dept;
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee bonus processed for ' || p_dept || ' department.');
```

END;

/

Execute : EXEC UpdateEmployeeBonus('Finance',8);

Output

Employee bonus processed for Finance department.

Scenario 3: Transfer Funds

CREATE OR REPLACE PROCEDURE TransferFunds

(fromAccount NUMBER, toAccount NUMBER, amount NUMBER)

AS

currentBalance NUMBER;

BEGIN

SELECT Balance

INTO currentBalance

FROM Accounts

WHERE AccountID = fromAccount;

IF currentBalance >= amount THEN

UPDATE Accounts

SET Balance = Balance - amount

WHERE AccountID = fromAccount;

UPDATE Accounts

SET Balance = Balance + amount

WHERE AccountID = toAccount;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Fund transfer completed.');

ELSE

DBMS_OUTPUT.PUT_LINE('Transaction cancelled due to low balance.');

END IF;

END;

/

Ececute : EXEC TransferFunds(1003,1004,2500);

Output

Fund transfer completed.

Insufficient Balance Case : EXEC TransferFunds(1003,1004,75000);

Output

Transaction cancelled due to low balance.